Détecteur de similarité entre codes sources

Rapport détaillé du projet

Comparaison textuelle, lexicale et structurelle

Python · Java · C · C++ · JavaScript

Élément	Détail
Langage principal	Python ≥ 3.10 (développé sous 3.14)
Architecture	MVC (Modèle / Vue / Contrôleur)
Interface graphique	Tkinter + matplotlib
Interface CLI	argparse (cli.py)
Moteur d'analyse	tree-sitter + algorithmes MOSS/JPlag
Tests	84 tests pytest (100 % réussite)
Licence / usage	Pédagogique (projet universitaire)

Avertissement éthique : un score de similarité ne constitue pas, à lui seul, une preuve de plagiat. Il signale des ressemblances statistiques et structurelles devant impérativement faire l'objet d'un examen critique par un humain qualifié.

Table des matières

1. Présentation du projet

2. Fonctionnalités

3. Architecture MVC

4. Installation

5. Utilisation de l'interface graphique

6. Utilisation de la ligne de commande (CLI)

7. Les trois niveaux de comparaison

7.1 Niveau 1 — Comparaison textuelle

7.2 Niveau 2 — Tokens et transformations

7.3 Niveau 3 — Structure inter-langages

8. Paramètres de configuration

9. Seuils et interprétation des scores

10. Structure des fichiers du projet

11. Tests automatisés

12. Évaluation des performances

13. Exécutables autonomes (Windows)

14. Exports CSV et HTML

15. Avertissement éthique

16. Auteur

1. Présentation du projet

Le **Détecteur de similarité entre codes sources** est un outil d'aide à la détection de ressemblances entre fichiers de code source, développé dans le cadre d'un projet universitaire. Il compare un fichier de référence à un ou plusieurs candidats et produit un score de similarité entre 0 et 100 %, accompagné de fragments surlignés, d'une analyse détaillée et d'exports CSV / HTML.

L'outil est conçu pour assister (et non remplacer) l'examen humain : il signale des ressemblances statistiques et structurelles que l'enseignant doit ensuite interpréter critiquement.

Trois niveaux de comparaison complémentaires sont proposés, allant de la copie brute à la reconnaissance de structures algorithmiques identiques dans des langages différents (Python, Java, C, C++, JavaScript).

2. Fonctionnalités

- **Trois niveaux de comparaison** complémentaires : textuel, tokens/transformations, structure inter-langages.
- **Interface graphique (Tkinter)** : sélection des fichiers, choix du niveau, tableau de résultats triable, visualisation matplotlib, vue de diff avec surlignage des fragments.
- **Interface en ligne de commande (cli.py)** : comparaison fichier par fichier ou par répertoire, niveau « auto » (niveau 2 pour un même langage, niveau 3 dès que les langages diffèrent — choix fait par couple référence/candidat), exports CSV et HTML.
- **Jeu d'essai (samples/)** : copies, reformatages, renommages, réordonnancements, cas limites et paires cross-langages.
- **Évaluation (evaluation/)** : scénarios étiquetés, métriques de classification (précision, rappel, F1), graphiques matplotlib.
- **Tests (tests/)** : 84 tests pytest couvrant le cœur, les trois niveaux et les exigences de la spécification.
- **Exécutables autonomes Windows** dans dist/ (PyInstaller).

3. Architecture MVC

Le projet suit une architecture **MVC (Modèle / Vue / Contrôleur)** stricte :

- **Modèle (models/)** — logique métier pure, sans aucune dépendance Tkinter.
- **Vue (views/)** — interface Tkinter et rendu console ; aucune logique métier.
- **Contrôleur (controllers/)** — orchestration modèle ↔ vue, threads, exports.

Arborescence principale :

Fichier / dossier	Rôle
main.py	Point d'entrée de l'interface graphique
cli.py	Point d'entrée de la ligne de commande
controllers/app_controller.py	Comparaison GUI, exports, ouverture du diff
controllers/cli_controller.py	Parcours complet de la CLI
models/config.py	Constantes, seuils, poids, couleurs

Fichier / dossier	Rôle
models/loader.py	Chargement multi-encodages, détection binaire/langage
models/lexer.py	Lexage tree-sitter + repli regex
models/normalizer.py	Normalisation textuelle et token, renommage
models/canonical.py	Catégories canoniques inter-langages
models/ast_parser.py	AST unifié, extraction de fonctions, complexité
models/fragments.py	Fragments de correspondance
models/ranking.py	Classement et niveaux d'alerte
models/methods/base.py	Protocole SimilarityMethod, SimilarityResult
models/methods/level1_text.py	Textuel (SequenceMatcher, Levenshtein, LCS…)
models/methods/level2_tokens.py	Winnowing, GST, appariement de fonctions
models/methods/level3_structure.py	AST unifié, édition d'arbre, motifs
models/report/export_csv.py	Export CSV
models/report/export_html.py	Export HTML
views/main_window.py	Fenêtre principale (en-tête, corps, statut)
views/control_panel.py	Barre latérale de configuration
views/results_view.py	Tableau classé + graphique
views/diff_view.py	Vue côte à côte surlignée
views/theme.py	Palette et styles ttk
views/console.py	Rendu texte de la CLI
evaluation/	Scénarios étiquetés, métriques, graphiques
tests/	Tests pytest (cœur, niveaux 1/2/3, GUI)
samples/	Jeu d'essai organisé par catégorie
docs/rapport.md	Rapport de projet

4. Installation

Prérequis : Python ≥ 3.10 (développé sous 3.14), accès réseau pour pip.

Dépendances : tree-sitter, tree-sitter-python, tree-sitter-java, tree-sitter-c, tree-sitter-cpp, tree-sitter-javascript, matplotlib, numpy, pytest.

Installation des dépendances

```
pip install -r requirements.txt
```

Lancement de l'interface graphique

```
python main.py
```

Lancement des tests

```
python -m pytest tests -q
```

5. Utilisation de l'interface graphique

1. Charger un fichier de référence (bouton « Parcourir » dans la section ①).
2. Ajouter un ou plusieurs candidats (fichier ou répertoire) dans la section ②.
3. Choisir le niveau (1, 2, 3 ou auto) dans la section ③.
4. Ajuster les paramètres avancés si besoin (section ④).
5. Lancer la comparaison (bouton bleu « ■ Comparer »).
6. Consulter le tableau de classement, la visualisation et le diff.
7. Exporter en CSV ou HTML via les boutons du bas.

La fenêtre principale présente :

- **En-tête** : titre, sous-titre, pastilles de seuils (faible / modéré / élevé).
- **Barre latérale gauche** : configuration (référence, candidats, niveau, paramètres), bouton Comparer, progression, exports.
- **Zone principale droite** : bandeau éthique, cartes de synthèse (candidats analysés, alertes élevées, score moyen, meilleur score), tableau classé, détail du candidat sélectionné, graphique à barres.

Un **double-clic** sur une ligne du tableau ouvre la vue côte à côte (diff) avec surlignage des fragments correspondants.

6. Utilisation de la ligne de commande (CLI)

Exemples d'utilisation :

```
python cli.py --ref samples/identical/bubble_sort_ref.py \ --cands
samples/modified/bubble_sort_renamed.py samples/unrelated/gcd.py \ --level 2

python cli.py --ref samples/identical/bubble_sort_ref.py \ --cands-dir
samples/cross_language --level auto \ --csv rapport.csv --html rapport.html
```

Options principales :

- **--level {1,2,3,auto}** : niveau de comparaison.
- **--kgram N** : taille des k-grammes (niveau 2).
- **--gst-min N** : longueur minimale d'une tuile GST.
- **--min-frag N** : longueur minimale d'un bloc commun (lignes).
- **--keep-comments** : conserver les commentaires.
- **--csv PATH** / **--html PATH** : exports.

7. Les trois niveaux de comparaison

L'outil propose trois niveaux complémentaires, du plus sensible au plus structurel, ainsi qu'un mode « auto » qui choisit le niveau par couple référence/candidat (niveau 2 si même langage, niveau 3 si langages différents).

7.1 Niveau 1 — Comparaison textuelle

Normalisation basique, ratio de lignes, Levenshtein, n-grammes, LCS. Sensible au formatage : repère les copies brutes.

Combinaison pondérée (total = 1,0) :

Composante	Poids	Description
sequence_matcher	0,35	Ratio difflib SequenceMatcher
levenshtein	0,25	Similarité de Levenshtein normalisée
jaccard_ngrams	0,25	Jaccard sur n-grammes de tokens textuels
lcs_lines	0,15	Plus longue sous-séquence commune (lignes)

Paramètres : n-grammes de taille 4, bloc commun minimum de 3 lignes.

7.2 Niveau 2 — Tokens et transformations

Lexage via tree-sitter, normalisation des identifiants / nombres / chaînes, winnowing (fenêtre glissante), Greedy String Tiling (GST), appariement de fonctions, détection de renommage bijectif. Robuste au renommage, au déplacement de fonctions et au reformatage.

Composante	Poids	Description
winnowing	0,30	Empreintes Winnowing (algorithme MOSS)
gst	0,35	Greedy String Tiling (algorithme JPlag)
function_matching	0,20	Appariement glouton des fonctions/méthodes
ast_subtrees	0,15	Hash des sous-arbres normalisés

Paramètres : k-grammes de taille 5, fenêtre de winnowing de 4, tuile GST minimale de 4 tokens.

Les identifiants des bibliothèques standards (print, len, System.out, printf, console.log, std::cout, …) ne sont pas renommés en « ID » afin de conserver l'information discriminante.

7.3 Niveau 3 — Structure inter-langages

AST unifié via tree-sitter, réduction aux nœuds canoniques (LOOP, IF, FUNCTION_DEF, …), séquences de k-grammes + LCS, distance de variation totale des descripteurs structurels, édition d'arbre simplifiée avec appariement glouton par catégorie, reconnaissance heuristique de motifs algorithmiques (tri à bulles, recherche binaire, fibonacci, PGCD, factorielle, primalité). Fonctionne entre Python, Java, C et JavaScript.

Composante	Poids	Description
canonical_sequence	0,50	K-grammes + LCS sur séquence de nœuds canoniques
structural_vector	0,20	Distance de variation totale des descripteurs
tree_similarity	0,15	Édition d'arbre réduit, appariement par catégorie
pattern_heuristics	0,15	Reconnaissance heuristique de motifs algorithmiques

La séquence canonique pèse le plus (mesure la plus discriminante entre algorithmes). Un facteur d'échelle multiplicatif doux de 1,15 compense la baisse mécanique des scores due aux particularités de syntaxe de chaque langage.

8. Paramètres de configuration

Paramètre	Défaut	Plage	Rôle
k-grammes (niv. 2)	5	2–12	Taille des k-grammes de tokens

Paramètre	Défaut	Plage	Rôle
Tuile min. (GST)	4	2–20	Longueur minimale d'une tuile GST
Bloc min. (lignes)	3	1–20	Longueur min d'un bloc commun
Seuil d'alerte (%)	70	0–100	Seuil au-dessus duquel l'alerte est élevée
Ignorer les commentaires	oui	—	Retire les commentaires avant analyse
Ignorer les imports	oui	—	Retire les lignes d'import/include

9. Seuils et interprétation des scores

Score	Niveau d'alerte	Signification indicative
< 30 %	Faible (vert)	Peu de ressemblances détectées
30–70 %	Modéré (orange)	Ressemblances notables
> 70 %	Élevé (rouge)	Fortes ressemblances

Le niveau 3 est conçu pour la comparaison inter-langages : sur des fichiers de même langage, préférer les niveaux 1 et 2, plus discriminants.

10. Structure des fichiers du projet

Voir la section 3 (tableau de l'architecture). Le projet contient également :

- **evaluation/** : scénarios étiquetés, métriques de classification (précision, rappel, F1), graphiques matplotlib.
- **samples/** : copies, reformatages, renommages, réordonnancements, cas limites et paires cross-langages.
- **dist/** : exécutables autonomes Windows (DetecteurDeSimilarite.exe, CLI-Similarite.exe).
- **docs/rapport.md** : rapport de projet détaillé.

11. Tests automatisés

84 tests pytest couvrant le cœur, les trois niveaux et les exigences de la spécification. Répartition :

Fichier	Contenu
test_core.py	Chargement, normalisation, classement, fragments
test_level1.py	Comparaison textuelle
test_level2.py	Tokens, winnowing, GST, fonctions
test_level3.py	Structure inter-langages, motifs
test_gui.py	Interface graphique (Tkinter)

```
python -m pytest tests -q
```

Résultat : 84 passed.

12. Évaluation des performances

```
python -m evaluation.run_evaluation # rapport texte python -m
evaluation.run_evaluation --csv resultats.csv --plots
```

Produit des métriques de classification (précision, rappel, F1) sur des scénarios étiquetés, avec graphiques matplotlib.

13. Exécutables autonomes (Windows)

Des exécutables autonomes sont fournis dans dist/ :

- **dist\DetecteurDeSimilarite** — version graphique ; lancer DetecteurDeSimilarite.exe (dossier samples/ joint pour démonstration).
- **dist\CLI-Similarite** — version console ; même usage que cli.py.

Pour reconstruire les exécutables après modification du code :

```
pip install pyinstaller py -m PyInstaller detecteur_similarite.spec --noconfirm #
version graphique py -m PyInstaller cli_similarite.spec --noconfirm # version
console
```

Les spécifications *.spec déclarent les grammaires tree-sitter comme imports cachés (chargées dynamiquement, sinon absentes du paquet).

14. Exports CSV et HTML

L'interface graphique et la CLI permettent d'exporter les résultats :

- **CSV** : tableau des candidats classés (rang, fichier, langage, score, alerte, fragments, temps).
- **HTML** : rapport complet avec tableau, détail des sous-scores, notes et visualisation.

15. Avertissement éthique

Un score de similarité ne constitue pas, à lui seul, une preuve de plagiat. Il signale des ressemblances statistiques et structurelles devant impérativement faire l'objet d'un examen critique par un humain qualifié.

16. Auteur

Projet réalisé dans le cadre d'un cours de programmation avancée — utilisation pédagogique.